```
"""
Project: Digital Exam and Results Management System
Module: PROG211 - Object-Oriented Programming 1
Domain: Education

This program uses fictional student data for demonstration.
"""


class Student:
    """Represents a student in the education system."""

    def __init__(self, student_id, name, course):
        self.student_id = student_id
        self.name = name
        self.course = course
        self.results = []

    def update_record(self, name=None, course=None):
        """Update the student's basic record."""
        if name is not None and name.strip():
            self.name = name.strip()

        if course is not None and course.strip():
            self.course = course.strip()

    def add_result(self, exam, mark):
        """Store a result linked to an Exam object."""
        if not exam.validate_mark(mark):
            return False

        # Update the result if this exam was already recorded.
        for result in self.results:
            if result["exam"].exam_id == exam.exam_id:
                result["mark"] = mark
                return True

        self.results.append({
            "exam": exam,
            "mark": mark
        })
        return True

    def display_info(self):
        """Display the student's information."""
        print(f"Student ID: {self.student_id}")
        print(f"Name: {self.name}")
        print(f"Course: {self.course}")


class Exam:
    """Represents an examination in a subject."""

    def __init__(self, exam_id, subject, maximum_mark=100):
        self.exam_id = exam_id
        self.subject = subject
        self.maximum_mark = maximum_mark

    def validate_mark(self, mark):
        """Check whether a mark is within the allowed range."""
        return (
            isinstance(mark, (int, float))
            and not isinstance(mark, bool)
            and 0 <= mark <= self.maximum_mark
        )

    def calculate_grade(self, mark):
        """Calculate a grade using the project's grading scale."""
        if not self.validate_mark(mark):
            return "Invalid mark"

        percentage = (mark / self.maximum_mark) * 100

        if percentage >= 80:
            return "A"
        elif percentage >= 70:
            return "B"
        elif percentage >= 60:
            return "C"
        elif percentage >= 50:
            return "D"
        else:
            return "F"

    def display_exam(self):
        print(f"Exam ID: {self.exam_id}")
        print(f"Subject: {self.subject}")
        print(f"Maximum Mark: {self.maximum_mark}")


class ResultsManagementSystem:
    """Manages student and examination objects."""

    def __init__(self):
        self.students = []
        self.exams = []

    def add_student(self, student):
        for existing in self.students:
            if existing.student_id == student.student_id:
                print("Error: Student ID already exists.")
                return False

        self.students.append(student)
        print("Student registered successfully.")
        return True

    def add_exam(self, exam):
        for existing in self.exams:
            if existing.exam_id == exam.exam_id:
                print("Error: Exam ID already exists.")
                return False

        if exam.maximum_mark <= 0:
            print("Error: Maximum mark must be positive.")
            return False

        self.exams.append(exam)
        print("Examination added successfully.")
        return True

    def find_student(self, student_id):
        for student in self.students:
            if student.student_id == student_id:
                return student
        return None

    def find_exam(self, exam_id):
        for exam in self.exams:
            if exam.exam_id == exam_id:
                return exam
        return None

    def record_result(self, student_id, exam_id, mark):
        student = self.find_student(student_id)
        exam = self.find_exam(exam_id)

        if student is None:
            print("Error: Student not found.")
            return False

        if exam is None:
            print("Error: Examination not found.")
            return False

        if not student.add_result(exam, mark):
            print("Error: Mark is outside the permitted range.")
            return False

        print("Result recorded or updated successfully.")
        return True

    def display_results(self, student_id):
        student = self.find_student(student_id)

        if student is None:
            print("Student not found.")
            return

        print("\n========== STUDENT RESULT ==========")
        student.display_info()

        if not student.results:
            print("No examination results recorded.")
            return

        for result in student.results:
            exam = result["exam"]
            mark = result["mark"]
            grade = exam.calculate_grade(mark)

            print("-------------------------------------")
            print(f"Subject: {exam.subject}")
            print(f"Mark: {mark}/{exam.maximum_mark}")
            print(f"Grade: {grade}")

        print("=====================================")


def main():
    # Create the management system object.
    system = ResultsManagementSystem()

    # Create Student objects.
    student1 = Student("ST001", "Mariama Kamara", "Computer Science")
    student2 = Student("ST002", "Abdul Sesay", "Information Technology")

    # Create Exam objects.
    exam1 = Exam("EX001", "Mathematics", 100)
    exam2 = Exam("EX002", "Programming", 100)

    # Interact with the objects through the system.
    system.add_student(student1)
    system.add_student(student2)

    system.add_exam(exam1)
    system.add_exam(exam2)

    # Record results by linking students to exam objects.
    system.record_result("ST001", "EX001", 78)
    system.record_result("ST001", "EX002", 86)
    system.record_result("ST002", "EX001", 65)

    # Display results.
    system.display_results("ST001")
    system.display_results("ST002")

    # Demonstrate updating a student's record.
    student1.update_record(course="Software Engineering")

    # Demonstrate updating an existing examination result.
    system.record_result("ST001", "EX001", 82)

    print("\nAfter updating the student record and result:")
    system.display_results("ST001")


if __name__ == "__main__":
    main()
```